OGC® DOCUMENT: 26-043

External identifier of this OGC® document: http://www.opengis.net/doc/IS/indoorjson/2.0

Open Geospatial Consortium

OGC INDOORGML 2.0 PART 2B – JSON ENCODING

STANDARD
Implementation

CANDIDATE SWG DRAFT

Version: 0.5.0
Submission Date: 2026-02-28
Approval Date: 2099-01-01
Publication Date: 2099-01-01
Editor: Taehoon Kim, Abdoulaye Diakite, Ki-Joune Li, Zhiyong Wang

License Agreement

Use of this document is subject to the license agreement at https://www.ogc.org/license

Suggested additions, changes and comments on this document are welcome and encouraged. Such suggestions may be submitted using the online change request form on OGC web site: http://ogc.standardstracker.org/

Copyright notice

Note

Attention is drawn to the possibility that some of the elements of this document may be the subject of patent rights. The Open Geospatial Consortium shall not be held responsible for identifying any or all such patent rights.

Recipients of this document are requested to submit, with their comments, notification of any relevant patent claims or other intellectual property rights of which they may be aware that might be infringed by any implementation of the standard set forth in this document, and to provide supporting documentation.

CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF NORMATIVE STATEMENTS

I ABSTRACT

The IndoorGML 2.0 Part 2b – JSON Encoding Standard presents the implementation-dependent JavaScript Object Notation (JSON) encoding of the concepts defined by the IndoorGML 2.0 Part 1 – Conceptual Model Standard (OGC 22-045r5). The JSON Encoding is compliant to JSON schema specified by RFC8259. The IndoorGML JSON (IndoorJSON) encoding defines a concrete implementation schema for representing indoor spatial information, including primal space, dual space, thematic layers, interlayer connections, and navigation-related indoor features. This encoding can be used to store and exchange 2D and/or 3D indoor space and indoor navigation network models in the JSON format as data sets or via web services.

This Part 2b of the Standard defines how the concepts developed in Part 1 are realized using JSON schemas. The IndoorJSON Encoding represents a full mapping of the Conceptual Model and is derived fully automatically from the UML model following the UML-to-JSON encoding rules as defined by OGC 24-017r1. The IndoorJSON adopts JSON Schema as its primary schema language and uses geometry objects compatible with OGC Features and Geometries JSON (JSON-FG, OGC 21-045r1), where appropriate. This design choice addresses the limitation that conventional GeoJSON does not provide sufficient support for indoor 3D geometry, especially solid and volumetric geometry. Table 1 maps requirements classes from the IndoorGML 2.0 Conceptual Model into the implementation details documented in this standard.

Table 1 — Conceptual Model Mapping

CONCEPTUAL MODEL	SECTION	JSON SCHEMA
IndoorGML Core	Clause 7	Annex B.1
Navigation Extension	Clause 8	Annex B.2

II KEYWORDS

The following are keywords to be used by search engines and document catalogues.

OGCDoc, OGC documents, indoor, navigation, IndoorGML, JSON, GeoJSON, JSON-FG

PREFACE

In order to achieve consensus on the modeling indoor spaces and their neighborhood relationships to support indoor location-based services, a UML Conceptual Model, IndoorGML 2.0 Part 1, was approved as an OGC Standard (OGC 22-045r5) in June 2025. This model covers geometric and semantic properties of indoor spaces relevant for indoor navigation. The IndoorGML 2.0 Part 2b: JSON Encoding (IndoorJSON) Standard defines how those concepts should be realized using JSON format. The IndoorJSON encoding is intended for software developers, data providers, standards implementers, and service providers who need a lightweight, web-friendly, and machine-readable representation of IndoorGML datasets.

SECURITY CONSIDERATIONS

No security considerations have been made for this Standard.

V SUBMITTING ORGANIZATIONS

The following organizations submitted this Document to the Open Geospatial Consortium (OGC):

- The University of New South Wales
- Pusan National University
- CityGeometrix
- National Institute of Advanced Industrial Science and Technology
- SpatialAlgebra Technology Limited

VI SUBMITTERS

All questions regarding this submission should be directed to the editor or the submitters:

NAME	AFFILIATION	EMAIL	OGC MEMBER
Taehoon KIM	National Institute of Advanced Industrial Science and Technology	kim.taehoon at aist.go.jp	Yes
Abdoulaye Diakite	CityGeometrix Pty Ltd	abdou at citygeometrix.com	Yes
Ki-Joune Li	Pusan National University	lik at pnu.edu	Yes
Sisi Zlatanova	The University of New South Wales	s.zlatanova at unsw.edu.au	Yes
Zhiyong Wang	SpatialAlgebra Technology Limited	zhiyongwang at spatialalgebra.tech	Yes

1

SCOPE

1 SCOPE

The OGC IndoorGML 2.0 Part 2b – JSON Encoding Standard defines a JSON encoding for IndoorGML 2.0 Part 1 — Conceptual Model. The encoding is referred to as IndoorJSON.

The IndoorJSON Standard specifies:

- JSON schema for IndooGML 2.0 core module;
- JSON schema for IndoorGML 2.0 navigation module;
- the geometry encoding rules based on JSON-FG geometry objects; and
- a statement to which IndoorJSON conformance classes a IndoorGML 2.0 conforms to.

IndoorJSON is an implementation encoding of IndoorGML 2.0 Part 1 — Conceptual Model. Therefore, IndoorJSON instances shall be interpreted according to the semantics defined in the IndoorGML 2.0 conceptual model.

2

CONFORMANCE

2 CONFORMANCE

This standard defines implementation specification which specifies how the IndoorGML 2.0 Conceptual Model should be implemented using JavaScript Object Notation (JSON). The Standardization Target for this standard is:

- Implementations of the IndoorGML 2.0 Conceptual Model using JSON encodings.

Conformance with this standard shall be checked using all the relevant tests specified in Annex A (normative) of this document. The framework, concepts, and methodology for testing, and the criteria to be achieved to claim conformance are specified in the OGC Compliance Testing Policies and Procedures and the OGC Compliance Testing web site.

In order to conform to this OGC® interface standard, a software implementation shall choose to implement:

- Any one of the conformance levels specified in Annex A (normative).

All requirements-classes and conformance-classes described in this document are owned by the standard(s) identified.

Table 2 — Conformance Class URIs

CONFORMANCE CLASS	URI
IndoorGML Core	http://www.opengis.net/spec/indoorgml-2/2.0/json/conf/core
Navigation Extension	http://www.opengis.net/spec/indoorgml-2/2.0/json/conf/navi

3 NORMATIVE REFERENCES

3 NORMATIVE REFERENCES

The following documents are referred to in the text in such a way that some or all of their content constitutes requirements of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

Johannes Echterhoff: OGC 24-017r1, *Best Practice for OGC — UML to JSON Encoding Rules*. Open Geospatial Consortium (2025). http://www.opengis.net/doc/BP/uml-to-json-encoding/1.0.

Sisi Zlatanova, Abdoulaye Diakite, Taehoon Kim, Ki-Joune Li: OGC 22-045r5, *OGC IndoorGML 2.0 Part 1 – Conceptual Model*. Open Geospatial Consortium (2025). http://www.opengis.net/doc/IS/indoorgml/2.0.

Clemens Portele, Panagiotis (Peter) A. Vretanos: OGC 21-045r1, *OGC Features and Geometries JSON — Part 1: Core*. Open Geospatial Consortium (2026). http://www.opengis.net/doc/IS/json-fg-1/1.0.

ISO: ISO 19101-1, *Geographic information — Reference model — Part 1: Fundamentals*. International Organization for Standardization, Geneva https://www.iso.org/standard/59164.html.

ISO: ISO 19107, *Geographic information — Spatial schema*. International Organization for Standardization, Geneva https://www.iso.org/standard/66175.html.

T. Bray (ed.): IETF RFC 8259, *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC Publisher (2017). https://www.rfc-editor.org/info/rfc8259.

4 TERMS AND DEFINITIONS

4 TERMS AND DEFINITIONS

This document uses the terms defined in OGC Policy Directive 49, which is based on the ISO/IEC Directives, Part 2, Rules for the structure and drafting of International Standards. In particular, the word “shall” (not “must”) is the verb form used to indicate a requirement to be strictly followed to conform to this document and OGC documents do not use the equivalent phrases in the ISO/IEC Directives, Part 2.

This document also uses terms defined in the OGC Standard for Modular specifications (OGC 08-131r3), also known as the ‘ModSpec’. The definitions of terms such as standard, specification, requirement, and conformance test are provided in the ModSpec.

For the purposes of this document, the following additional terms and definitions apply.

4.1. conceptual model

model that defines concepts of a universe of discourse

[**SOURCE:** ISO 19101-1]

4.2. conceptual schema

formal description of a conceptual model

[**SOURCE:** ISO 19101-1]

4.3. feature

abstraction of real world phenomena

Note 1 to entry: Features are objects that have an identity that allows for distinguishing features from each other. Features can have spatial and non-spatial properties that describe the features in more detail. Spatial properties are properties that have a geometry from ISO 19107 as data type.

Note 2 to entry: Features are denoted by the stereotype «FeatureType» in the IndoorGML 2.0 Conceptual Model.

[**SOURCE:** ISO 19101-1]

5

CONVENTIONS

5 CONVENTIONS

5.1. Symbols (and abbreviated terms)

The following symbols and abbreviated terms are used in this standard. Symbols and abbreviated terms defined in IndoorGML 2.0 Part 1 also apply to this document.

Table 3 — Symbols and abbreviated terms

ABBREVIATION	WORD OR PHRASE
IndoorGML	Indoor Geographic Markup Language
IndoorJSON	JSON encoding of IndoorGML concepts
JSON	JavaScript Object Notation
OGC	Open Geospatial Consortium
UML	Unified Modeling Language
1D	One Dimensional
2D	Two Dimensional
3D	Three Dimensional

5.2. Identifiers

The normative provisions in this standard are denoted by the URI

`http://www.opengis.net/spec/indoorgml-2/2.0/json`

All requirements and conformance tests that appear in this document are denoted by partial URIs which are relative to this base.

5.3. Use of JSON terminology

The following terms are used as specified in JSON. In particular:

- “member” refers to a name/value pair of an object;
- “key” refers to the name of a member; and
- “value” refers to the value of a member, which may be an object, array, string, number, boolean, or `null`.

For example, the GeoJSON “geometry” member has the key “geometry” and one of the GeoJSON geometries (or `null`) as its value.

6

OVERVIEW OF THE JSON ENCODING

6 OVERVIEW OF THE JSON ENCODING

The JavaScript Object Notation (JSON) encoding of IndoorGML 2.0 (IndoorJSON) implements the conceptual classes of IndoorGML 2.0 Part 1 as JSON objects. The conceptual model remains normative for semantics, while this document defines the JSON representation.

6.1. Design Principle

IndoorJSON is based on the following encoding principles.

6.1.1. Conceptual model preservation

IndoorJSON Core preserve the IndoorGML 2.0 core conceptual model structure, as shown in the below figure.

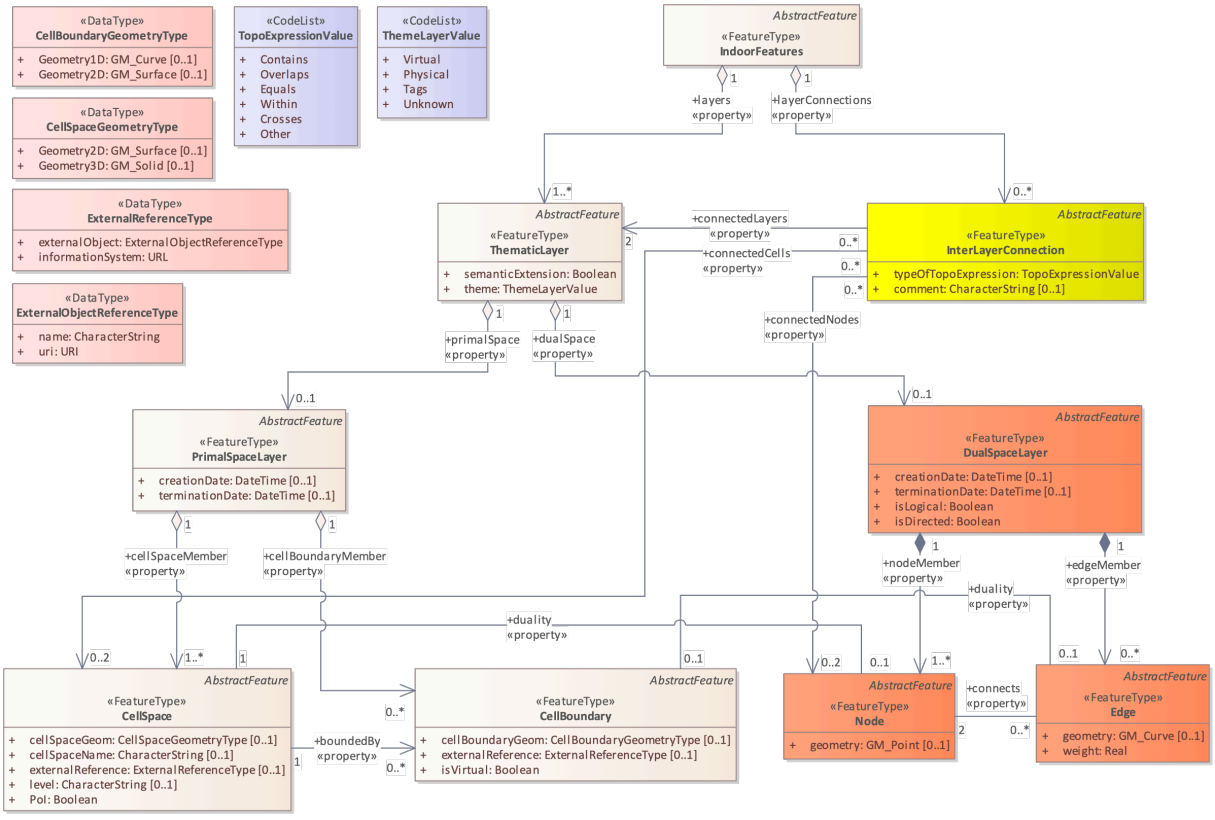

Figure 1 — IndoorGML 2.0 core module structure UML

The IndoorJSON Core schema follows Figure 1. Each class is defined individually, and depending on its relationships, it can hold other classes' IDs as property values or array values. For example,

IndoorFeatures have `layers` property for **ThematicLayer** and `layerConnections` property for **InterLayerConnection**.

IndoorJSON Navigation preserve the IndoorGML 2.0 navigation conceptual model structure, as shown in the below figure.

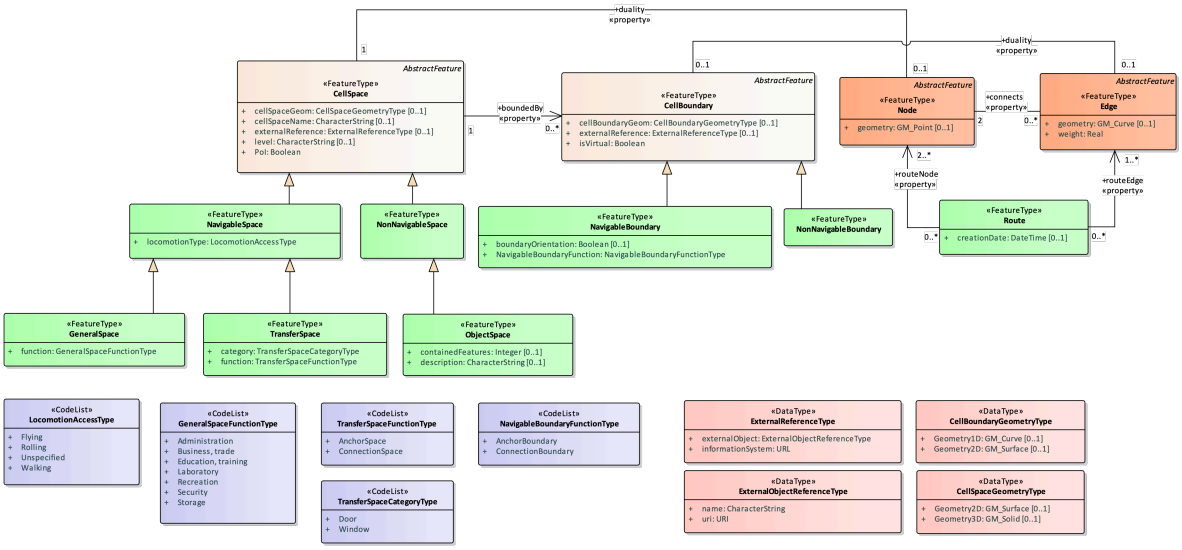

Figure 2 — IndoorGML 2.0 navigation module structure UML

The IndoorJSON Navigation schema follows Figure 2. Each class is defined individually. Depending on the inheritance relationship, a child class inherits the properties of its parent class.

6.1.2. JSON-FG-compatible geometry

IndoorJSON use JSON-FG-compatible geometry objects for the geometry representation. This is necessary because indoor spaces require 3D geometry, including solid geometry. The current IndoorJSON core schema references JSON-FG geometry schemas for `geometry2D` and `geometry3D`, including **Polygon** and **Polyhedron** for `CellSpace`.

7 INDOORJSON CORE

7 INDOORJSON CORE

REQUIREMENTS CLASS 1	
IDENTIFIER	`http://www.opengis.net/spec/indoorgml-2/2.0/json/req/core`
TARGET TYPE	Implementation Specification
CONFORMANCE CLASS	Conformance class A.1: `http://www.opengis.net/spec/indoorgml-2/2.0/conf/core`
PREREQUISITES	http://www.opengis.net/spec/indoorgml/2.0/req http://www.opengis.net/spec/json-fg-1/1.0/req/core http://www.opengis.net/spec/json-fg-1/1.0/req/polyhedra
NORMATIVE STATEMENTS	Requirement 1: `/req/core/indoorjson` Requirement 2: `/req/core/element` Requirement 3: `/req/core/indoorfeature` Requirement 4: `/req/core/thematiclayer` Requirement 5: `/req/core/primalspacelayer` Requirement 6: `/req/core/cellspace` Requirement 7: `/req/core/cellboundary` Requirement 8: `/req/core/dualspacelayer` Requirement 9: `/req/core/node` Requirement 10: `/req/core/edge` Requirement 11: `/req/core/interlayerconnection`

7.1. General requirements for IndoorJSON

IndoorJSON object represents one of feature class of IndoorGML 2.0 conceptual model.

The IndoorJSON Object:

- **must** have one member with the name `id`. The value must be a string that serves as a unique identifier for the IndoorJSON object and can be used to represent cross-references between IndoorJSON objects.
- **must** have one member with the name `featuretype`. The value is one of the possibilities in the Figure 1 or Figure 2.

REQUIREMENT 1: REQUIREMENT FOR INDOORJSON OBJECT	
IDENTIFIER	`/req/core/indoorjson`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req/core`
A	An IndoorJSON object SHALL contain `id` and `featuretype` members.

REQUIREMENT 2: REQUIREMENT FOR INDOORJSON CORE	
IDENTIFIER	`/req/core/element`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req/core`
A	IndoorGML Core JSON elements implemented by a conforming instance document SHALL conform to the JSON schema in indoorjson.core.schema.json.

7.2. IndoorFeatures

`IndoorFeature` object is an instance of IndoorFeature, the container for the indoor dataset. It aggregates one or more thematic layers and may include interlayer connections.

The **`IndoorFeature`** object shall contain:

- `id`
- `featureType`
- `layers` for representing **`ThematicLayer`** object as item
- optionally, `layerConnections` for **`InterLayerConnection`** object as item

The following examples refer to an **`IndoorFeature`** object shown in Figure 3. In the example, the **`IndoorFeature`** object contains an array of **`ThematicLayer`** objects as the value of its `layers` member and an array of **`InterLayerConnection`** objects as the value of its `interlayerConnections` member. Each object has an `id` member, which is used by specific members (such as `connectedLayers` or `connectedNodes` in **`InterLayerConnection`**) to reference the object.

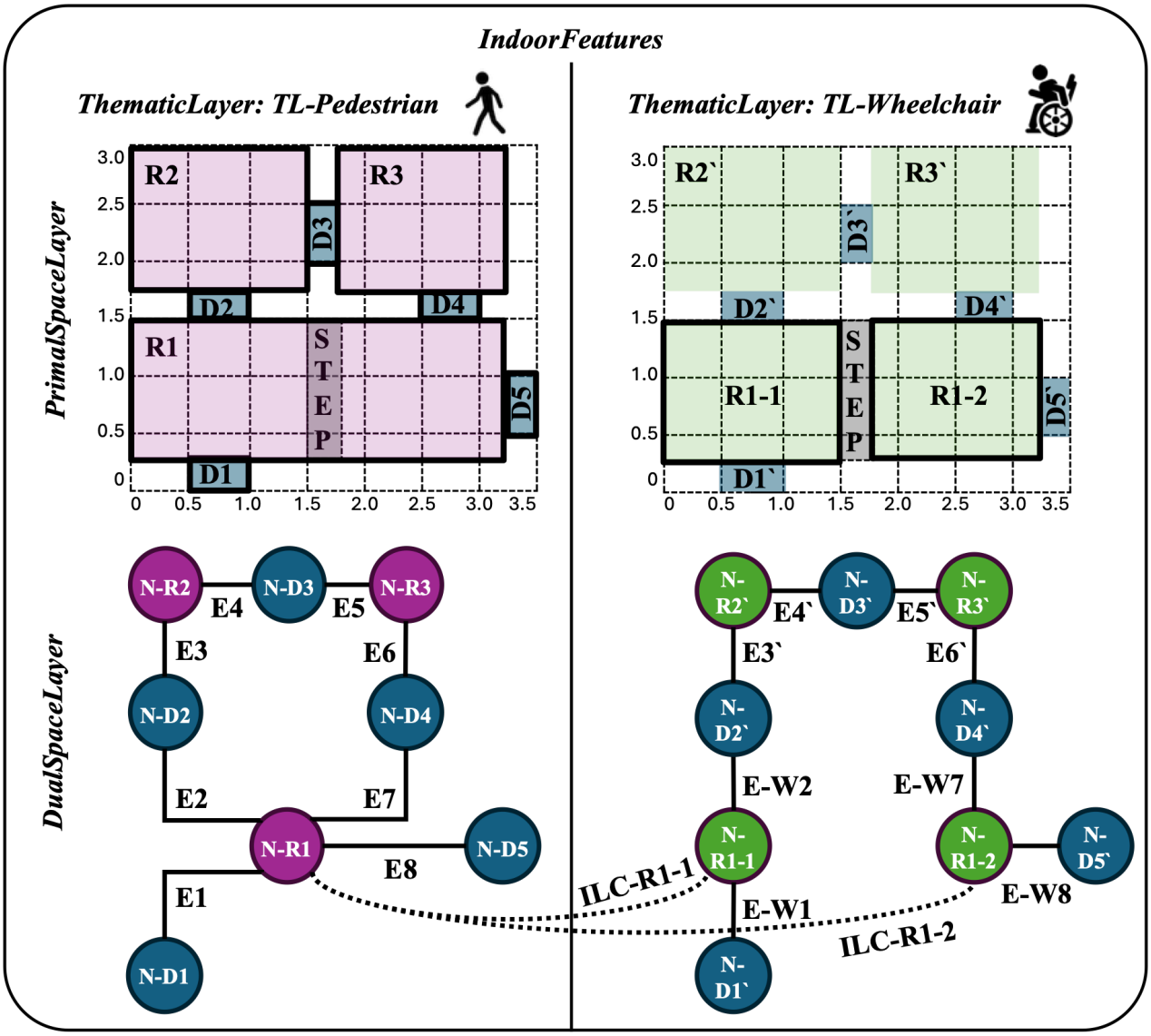

Figure 3 — IndoorJSON example layout

```
{
  "IndoorFeatures": {
    "$anchor": "IndoorFeatures",
    "type": "object",
    "properties": {
      "id": { "type": "string" },
      "featureType": { "enum": ["IndoorFeatures"] },
      "layers": {
        "type": "array",
        "minItems": 1,
        "items": { "$ref": "#/$defs/ThematicLayer" },
        "uniqueItems": true
      },
      "layerConnections": {
        "type": "array",
        "items": { "$ref": "#/$defs/InterLayerConnection" },
        "uniqueItems": true
      }
    },
```

```
    "required": ["id","featureType","layers"]
  }
}
```

Listing 1 — A JSON schema of the IndoorFeature

```
{
  "id": "IFs",
  "featureType": "IndoorFeatures",
  "layers": [
    {
      "id": "TL-Pedestrian",
      "featureType": "ThematicLayer",
      "semanticExtension": false,
      "theme": "Physical",
      "primalSpace": {...},
      "dualSpace": {...}
    },
    ...
  ],
  "layerConnections": [
    {
      "id": "ILC-1",
      "featureType": "InterLayerConnection",
      "connectedLayers": ["TL-Pedestrian", "TL-Wheelchair"],
      "connectedCells": ["R1", "R1-1"],
      "connectedNodes": ["N-R1", "N-R1-1"],
      "typeOfTopoExpression": "CONTAINS"
    },
    ...
  ]
}
```

Listing 2 — An example of the IndoorFeature object

REQUIREMENT 3: INDOORFEATURES SCHEMA	
IDENTIFIER	`/req/core/indoorfeature`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req/core`
A	An **`IndoorFeatures`** object SHALL contain at least one **`ThematicLayer`** object as item of `layers` value.
B	The `featuretype` value SHALL be “IndoorFeatures”.

7.3. ThematicLayer

`ThematicLayer` object is an instance of ThematicLayer, which represents one contextual decomposition of indoor space. For example, physical layer, navigation layer, sensor layer, etc.

The **ThematicLayer** object shall contain:

- id
- featureType
- semanticExtention indicates whether there is extension module with additional semantic information associated to the **PrimalSpaceLayer**
- theme determines what type of model representation can be expected in the **PrimalSpaceLayer**
- optionally, primalSpace for **PrimalSpaceLayer** object
- optionally, dualSpace for **DualSpaceLayer** object

The core schema defines semanticExtension and theme members, with theme values including "Virtual", "Physical", "Tags", and "Unknown", according to the conceptual model.

```
{
  "ThematicLayer": {
    "$anchor": "ThematicLayer",
    "type": "object",
    "properties": {
      "id": { "type": "string" },
      "featureType": { "enum": ["ThematicLayer"] },
      "semanticExtension": { "type": "boolean" },
      "theme": { "enum": ["Virtual", "Physical", "Tags", "Unknown"] },
      "primalSpace": { "$ref": "#/$defs/PrimalSpaceLayer" },
      "dualSpace": { "$ref": "#/$defs/DualSpaceLayer" }
    },
    "required": ["id","featureType","semanticExtension","theme"]
  }
}
```

Listing 3 — A JSON schema of the ThematicLayer

The following examples refer to a **ThematicLayer** object ("TL-Pedestrian"), shown in Figure 3.

```
{
  "id": "TL-Pedestrian",
  "featureType": "ThematicLayer",
  "semanticExtension": false,
  "theme": "Physical",
  "primalSpace": {
    "id": "PS-1",
    "featureType": "PrimalSpaceLayer",
    "creationDatetime": "2025-10-30T13:00:00",
    "cellSpaceMember": [...],
    "cellBoundaryMember": [...]
  },
  "dualSpace": {
    "id": "DS-1",
    "featureType": "DualSpaceLayer",
    "creationDatetime": "2025-10-30T13:00:00",
    "isLogical": false,
    "isDirected": false,
    "nodeMember": [...],
    "edgeMember": [...]
```

```
  }
}
```

Listing 4 — An example of the ThematicLayer object

REQUIREMENT 4: THEMATICLAYER SCHEMA	
IDENTIFIER	`/req/core/thematiclayer`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req/core`
A	A **`ThematicLayer`** object SHALL declare whether there is an extension module with additional semantic information using the `semanticExtension` member.
B	A **`ThematicLayer`** object SHALL determine what type of model representation can be expected using the `theme` member.
C	The `Theme` value SHALL be one of the following: “Virtual”, “Physical”, “Tags”, or “Unknown”
D	The `featuretype` value SHALL be “ThematicLayer”.

7.4. PrimalSpaceLayer

`PrimalSpaceLayer` object is an instance of PrimalSpaceLayer, which contains spatial objects representing indoor subdivisions.

The **`PrimalSpaceLayer`** object shall contain:

- `id`
- `featureType`
- `cellSpaceMember` for representing **`CellSpace`** object as item
- optionally, `cellBoundaryMember` for representing **`CellSBoundary`** object as item
- optionally, `creationDatetime`
- optionally, `terminationDatetime`

```
{
  "PrimalSpaceLayer": {
    "$anchor": "PrimalSpaceLayer",
    "type": "object",
    "properties": {
      "id": { "type": "string" },
      "featureType": { "enum": ["PrimalSpaceLayer"] },
      "creationDatetime": { "type": "string", "format": "date-time" },
      "terminationDatetime": { "type": "string", "format": "date-time" },
```

```
      "cellSpaceMember": {
        "type": "array",
        "minItems": 1,
        "items": { "$ref": "#/$defs/CellSpace" },
        "uniqueItems": true
      },
      "cellBoundaryMember": {
        "type": "array",
        "items": { "$ref": "#/$defs/CellBoundary" },
        "uniqueItems": true
      }
    },
    "required": ["id","featureType","cellSpaceMember"]
  }
}
```

Listing 5 — A JSON schema of the PrimalSpaceLayer

The following examples refer to a **`PrimalSpaceLayer`** object of "TL-Pedestrian", shown in Figure 3.

```
{
  "primalSpace": {
    "id": "PS-1",
    "featureType": "PrimalSpaceLayer",
    "creationDatetime": "2025-10-30T13:00:00",
    "cellSpaceMember": [
      {
        "id": "R1",
        "featureType": "CellSpace",
        "duality": "N-R1",
        "poi": false,
        "level": "1",
        "cellSpaceGeom": {
          "geometry2D": {
            "type": "Polygon",
            "coordinates": [[[0,0.25], [3.25,0.25], [3.25,1.5], [0,1.5], [0,
0.25]]]
          }
        },
        "boundedBy": ["B1", "B2", "B7", "B8"]
      },
      ...
    ],
    "cellBoundaryMember": [
      {
        "id": "B1",
        "featureType": "CellBoundary",
        "isVirtual": false,
        "duality": "E1",
        "cellBoundaryGeom": {
          "geometry1D": {
            "type": "LineString",
            "coordinates": [[0.5,0.25],[1.0,0.25]]
          }
        }
      },
      ...
    ]
  }
```

```
}
```

Listing 6 — An example of the PrimalSpaceLayer object

REQUIREMENT 5: PRIMALSPACELAYER SCHEMA	
IDENTIFIER	`/req/core/primalspacelayer`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req/core`
A	A **`PrimalSpaceLayer`** SHALL contain one or more **`CellSpace`** objects.
B	The **`CellSpace`** objects belonging to the same **`PrimalSpaceLayer`** SHALL not intersect with each other.
C	The **`CellBoundary`** objects belonging to the same **`PrimalSpaceLayer`** SHALL not intersect with each other.
D	The `featuretype` value SHALL be “PrimalSpaceLayer”.

7.5. CellSpace

`CellSpace` object is an instance of CellSpace, which represents a unit of indoor space. The **`CellSpace`** may correspond to a room, corridor, stairwell, elevator cabin, platform, zone, virtual partition, or other spatial subdivision.

An **`CellSpace`** object shall contain:

- `id`
- `featureType`
- `poi` is a flag that specifies whether this **`CellSpace`** is a point of interset
- optionally, `cellSpaceName` is the name assigned to the space according to internal convention
- optionally, `level`
- optionally, `duality` can be mapped to the `id` value of the **`Node`** object
- optionally, `cellSpaceGeom`
- optionally, `boundedBy` can have the `id` value of the linked **`CellBoundary`** objects
- optionally, `externalReference` for representing **`ExternalReference`** object

A **CellSpace** can be linked to a **Node** through the duality member. A **CellSpace** can be linked to a set of **CellBoundaries** through the boundedBy member. For cellsSpaceGeom member, it defines cellSpaceGeom.geometry2D using a JSON-FG Polygon geometry and cellSpaceGeom.geometry3D using a JSON-FG Polyhedron geometry. The externalReference member is used for the reference of an object to its corresponding object in an external data set.

```
{
  "CellSpace": {
    "$anchor": "CellSpace",
    "type": "object",
    "properties": {
      "id": { "type": "string" },
      "featureType": { "enum": ["CellSpace",
        "NavigableSpace","GeneralSpace","TransferSpace",
        "NonNavigableSpace","ObjectSpace"] },
      "cellSpaceName:": { "type": "string" },
      "level": { "type": "string" },
      "poi": { "type":  "boolean" },
      "duality" : { "type": "string", "format": "uri-reference" },
      "cellSpaceGeom" : {
        "type": "object",
        "properties": {
          "geometry2D": { "$ref": "https://schemas.opengis.net/json-fg/geometry-
object.json#/$defs/Polygon" },
          "geometry3D": { "$ref": "https://schemas.opengis.net/json-fg/geometry-
object.json#/$defs/Polyhedron" }
        }
      },
      "boundedBy": {
        "type": "array",
        "items": { "type": "string", "format": "uri-reference" },
        "uniqueItems": true
      },
      "externalReference": { "$ref": "#/$defs/ExternalReferenceType" }
    },
    "required": ["id","featureType","poi"]
  }
}
```

Listing 7 — A JSON schema of the CellSpace

The following examples refer to a **CellSpace** object ("R1"), shown in Figure 3.

```
{
  "id": "R1",
  "featureType": "CellSpace",
  "duality": "N-R1",
  "poi": false,
  "level": "1",
  "cellSpaceGeom": {
    "geometry2D": {
      "type": "Polygon",
      "coordinates": [[[0,0.25], [3.25,0.25], [3.25,1.5], [0,1.5], [0,0.25]]]
    }
  },
  "boundedBy": ["B1", "B2", "B7", "B8"]
}
```

Listing 8 — An example of the CellSpace object

REQUIREMENT 6: CELLSPACE SCHEMA	
IDENTIFIER	`/req/core/cellspace`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req/core`
A	A **`CellSpace`** object SHALL declare whether it is a point of interest space using the `poi` member.
B	The `duality` value SHALL be the same as the `id` of the linked **`Node`** object.
C	The `boundedBy` value SHALL contain the same `id` as the linked **`CellBoundary`** objects.
D	The geometry of the linked **`CellBoundary`** SHALL not exceed the extent of the **`CellSpace`** geometry.
E	The `featuretype` value SHALL be one of “CellSpace”, “NonNavigableSpace”, or “ObjectSpace”.

7.6. CellBoundary

`CellBoundary` object is an instance of CellBoundary, a boundary of a CellSpace. A boundary may be physical or virtual.

The **`CellBoundary`** object shall contain:

- `id`
- `featureType`
- `isVirtual` indicate whether a **`CellBoundary`** corresponds to a virtual surface (true) or a physical one (false)
- optionally, `duality` can be mapped to the `id` value of the **`Edge`** object
- optionally, `cellBoundaryGeom`
- optionally, `externalReference` for representing **`ExternalReference`** object

A **`CellBoundary`** can be linked to a **`Edge`** through the `duality` member. For `cellBoundaryGeom` member, it defines `cellBoundaryGeom.geometry2D` using a `JSON-FG LineString` geometry and `cellBoundaryGeom.geometry3D` using a `JSON-FG Polygon` geometry. The `externalReference` member is used for the reference of an object to its corresponding object in an external data set

```
{
  "CellBoundary": {
    "$anchor": "CellBoundary",
    "type": "object",
    "properties": {
```

```
      "id": { "type": "string" },
      "featureType": { "enum": ["CellBoundary","NavigableBoundary",
"NonNavigableBoundary"] },
      "isVirtual": { "type":  "boolean" },
      "duality" : { "type": "string", "format": "uri-reference" },
      "cellBoundaryGeom" : {
        "type": "object",
        "properties": {
          "geometry1D": { "$ref": "https://schemas.opengis.net/json-fg/geometry-
object.json#/$defs/LineString" },
          "geometry2D": { "$ref": "https://schemas.opengis.net/json-fg/geometry-
object.json#/$defs/Polygon" }
        }
      },
      "externalReference": { "$ref": "#/$defs/ExternalReferenceType" }
    },
    "required": ["id","featureType","isVirtual"]
  }
}
```

Listing 9 — A JSON schema of the CellBoundary

The following examples refer to a **`CellBoundary`** object ("B1"), shown in Figure 3.

```
{
  "id": "B1",
  "featureType": "CellBoundary",
  "isVirtual": false,
  "duality": "E1",
  "cellBoundaryGeom": {
    "geometry1D": {
      "type": "LineString",
      "coordinates": [[0.5,0.25],[1.0,0.25]]
    }
  }
}
```

Listing 10 — An example of the CellBoundary object

REQUIREMENT 7: CELLBOUNDARY SCHEMA	
IDENTIFIER	`/req/core/cellboundary`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req/core`
A	A **`CellBoundary`** object SHALL specify whether it is virtual using the `isVirtual` member.
B	The `duality` value SHALL be the same as the `id` of the linked **Edge** object.
C	The `featuretype` value SHALL be one of "CellBoundary", or "NonNavigableBoundary".

7.7. DualSpaceLayer

`DualSpaceLayer` object is an instance of DualSpaceLayer, represents the graph structure corresponding to the primal space layer. It contains nodes and edges.

The **`DualSpaceLayer`** object shall contain:

- `id`
- `featureType`
- `isLogical` indicates whether the provided graph is a geometric or a logical
- `isDirected` specifies whether the provided graph is directed
- `nodeMember` for representing **`Node`** object as item
- optionally, `edgeMember` for representing **`Edge`** object as item
- optionally, `creationDatetime`
- optionally, `terminationDatetime`

```
{
  "DualSpaceLayer": {
    "$anchor": "DualSpaceLayer",
    "type": "object",
    "properties": {
      "id": { "type": "string" },
      "featureType": { "enum": ["DualSpaceLayer"] },
      "creationDatetime": { "type": "string", "format": "date-time" },
      "terminationDatetime": { "type": "string", "format": "date-time" },
      "isLogical": { "type": "boolean" },
      "isDirected": { "type": "boolean" },
      "nodeMember": {
        "type": "array",
        "minItems": 1,
        "items": { "$ref": "#/$defs/Node" },
        "uniqueItems": true
      },
      "edgeMember": {
        "type": "array",
        "items": { "$ref": "#/$defs/Edge" },
        "uniqueItems": true
      }
    },
    "required": ["id","featureType","isLogical","isDirected","nodeMember"]
  }
}
```

Listing 11 — A JSON schema of the DualSpaceLayer

The following examples refer to a **`DualSpaceLayer`** object of “TL-Pedestrian”, shown in Figure 3.

```
{
  "dualSpace": {
```

```
    "id": "DS-1",
    "featureType": "DualSpaceLayer",
    "creationDatetime": "2025-10-30T13:00:00",
    "isLogical": false,
    "isDirected": false,
    "nodeMember": [
      {
        "id": "N-R1",
        "featureType": "Node",
        "duality": "R1",
        "geometry": {
          "type": "Point",
          "coordinates": [1.625,0.875]
        },
        "connects": ["E1","E2","E7","E8"]
      },
      ...
    ],
    "edgeMember": [
      {
        "id": "E1",
        "featureType": "Edge",
        "duality": "B1",
        "weight": 1.0,
        "geometry": {
          "type": "LineString",
          "coordinates": [[0.75,0.125],[0.75,0.875],[1.625,0.875]]
        },
        "connects": ["N-D1", "N-R1"]
      },
      ...
    ]
  }
}
```

Listing 12 — An example of the DualSpaceLayer object

REQUIREMENT 8: DUALSPACELAYER SCHEMA	
IDENTIFIER	`/req/core/dualspacelayer`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req/core`
A	A **`DualSpaceLayer`** object SHALL specify whether the provided graph is a geometric or a logical using the `isLogical` member.
B	When the `isLogical` value is FALSE, the geometries of its **`Node`** instances SHALL be spatially located inside of their corresponding **`CellSpaces`**.
C	A **`DualSpaceLayer`** object SHALL specify whether the provided graph is directed using the `isDirected` member.
D	A **`DualSpaceLayer`** object SHALL contain one or more **`Node`** objects as `NodeMember` values.
E	The `featuretype` value SHALL be “DualSpaceLayer”.

7.8. Node

Node object is an instance of Node, a state or a graph vertex in the dual space layer.

The **Node** object shall contain:

- `id`
- `featureType`
- `duality` must be mapped to the `id` member of the **CellSpace** object
- optionally, `geometry`
- optionally, `connects` can have the `id` member of the connected **Edge** object

A **Node** may be linked to a **CellSpace** through the `duality` member. A **Node** may be connects to a set of **Edges** through the `connects` member. For `geometry` member referred `JSON-FG Point` geometry.

```
{
  "Node": {
    "$anchor": "Node",
    "type": "object",
    "properties": {
      "id": { "type": "string" },
      "featureType": { "enum": ["Node"] },
      "duality" : { "type": "string", "format": "uri-reference" },
      "geometry" : { "$ref": "https://schemas.opengis.net/json-fg/geometry-
object.json#/$defs/Point" },
      "connects" : {
        "type": "array",
        "items": { "type": "string", "format": "uri-reference" },
        "uniqueItems": true
      }
    },
    "required": ["id","featureType","duality"]
  }
}
```

Listing 13 — A JSON schema of the Node

The following examples refer to a **Node** object ("N-R1"), shown in Figure 3.

```
{
  "id": "N-R1",
  "featureType": "Node",
  "duality": "R1",
  "geometry": {
    "type": "Point",
    "coordinates": [1.625,0.875]
  },
  "connects": ["E1","E2","E7","E8"]
```

```
}
```

Listing 14 — An example of the Node object

REQUIREMENT 9: NODE SCHEMA	
IDENTIFIER	`/req/core/node`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req/core`
A	A **`Node`** object SHALL contain a `duality` value by referencing the `id` member of to correspond **`CellSpace`**.
B	The `connects` value SHALL contain the same `id` as the linked **`Edge`** objects.
C	The `featuretype` value SHALL be “Node”.

7.9. Edge

`Edge` object is an instance of Edge, which represents a relationship between two nodes. It may represent adjacency, connectivity, accessibility, or navigability.

The `Edge` object shall contain:

- `id`
- `featureType`
- `weight` can be used in graph-based applications
- `connects` must have the `id` value of the connected **`Nodes`** object
- optionally, `duality` can be mapped to the `id` value of the **`CellBoundary`** object
- optionally, `geometry`

An `Edge` must reference the connected **`Nodes`** using the `connects` member. An `Edge` can be linked to a **`CellBoundary`** through the `duality` member. For `geometry` member referred `JSON-FG LineString` geometry.

```
{
  "Edge": {
    "$anchor": "Edge",
    "type": "object",
    "properties": {
      "id": { "type": "string" },
      "featureType": { "enum": ["Edge"] },
      "duality" : { "type": "string", "format": "uri-reference" },
```

```
      "weight" : { "type": "number" },
      "geometry" : { "$ref": "https://schemas.opengis.net/json-fg/geometry-
object.json#/$defs/LineString" },
      "connects" : {
        "type": "array",
        "items": { "type": "string", "format": "uri-reference" },
        "minItems": 2,
        "maximum": 2,
        "uniqueItems": true
      }
    },
    "required": ["id","featureType","weight","connects"]
  }
}
```

Listing 15 — A JSON schema of an Edge

The following examples refer to a **Edge** object ("E1"), shown in Figure 3.

```
{
  "id": "E1",
  "featureType": "Edge",
  "duality": "B1",
  "weight": 1.0,
  "geometry": {
    "type": "LineString",
    "coordinates": [[0.75,0.125],[0.75,0.875],[1.625,0.875]]
  },
  "connects": ["N-D1", "N-R1"]
}
```

Listing 16 — An example of the Edge object

REQUIREMENT 10: EDGE SCHEMA	
IDENTIFIER	`/req/core/edge`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req/core`
A	An **Edge** object SHALL reference the connected **Nodes** using the `connects` member.
B	An **Edge** object SHALL contain a `weight` value.
C	The start and end coordinates of the `geometry` value SHALL match the coordinates of the connected **Nodes** in the `connects` member.
D	The `featuretype` value SHALL be "Edge".

7.10. InterLayerConnection

`InterLayerConnection` object is an instance of InterLayerConnection, represents a relationship between objects in different thematic layers.

The **`InterLayerConnection`** object shall contain:

- `id`
- `featureType`
- `connectedLayers` must be mapped to the `id` value of the two **`ThematicLayer`** objects
- `typeOfTopoExpression`
- optionally, `connectedNodes` can be mapped to the `id` value of the two **`Node`** objects
- optionally, `connectedCells` can be mapped to the `id` value of the two **`CellSpace`** objects
- optionally, `comment`

The `typeOfTopoExpression` member represents the topological relationship between two layers. The `typeOfTopoExpression` values including “CONTAINS”, “OVERLAPS”, “EQUALS”, “WITHIN”, “CROSSES”, and “OTHERS”, according to the conceptual model. Those topological values are in the form of verbs for which the subject is the first instance of the `connectedLayers` member. The values of the `connectedLayers`, `connectedNodes`, and `connectedCells` members shall be ordered and mapped to the same indices.

```
{
  "InterLayerConnection": {
    "$anchor": "InterLayerConnection",
    "type": "object",
    "properties": {
      "id": { "type": "string" },
      "featureType": { "enum": ["InterLayerConnection"] },
      "connectedLayers": {
        "type": "array",
        "items": { "type": "string", "format": "uri-reference" },
        "minItems": 2,
        "maximum": 2,
        "uniqueItems": true
      },
      "connectedNodes" : {
        "type": "array",
        "items": { "type": "string", "format": "uri-reference" },
        "minItems": 2,
        "maximum": 2,
        "uniqueItems": true
      },
      "connectedCells" : {
        "type": "array",
        "items": { "type": "string", "format": "uri-reference" },
        "minItems": 2,
        "maximum": 2,
        "uniqueItems": true
```

```
      },
      "typeOfTopoExpression" : {
        "enum": ["CONTAINS", "OVERLAPS", "EQUALS", "WITHIN", "CROSSES", "OTHERS"]
      },
      "comment" : { "type": "string" }
    },
    "required": ["id","featureType","connectedLayers","typeOfTopoExpression"]
  }
}
```

Listing 17 — A JSON schema of the InterLayerConnection

The following examples refer to a **`InterLayerConnection`** object ("ILC-R1-1"), shown in Figure 3. For example, consider a cell space ("R1") containing a step that pedestrians can traverse but that is difficult for wheelchairs to cross. If this distinction leads to the creation of separate thematic layers for pedestrians and wheelchairs, the cell space "R1" is split into "R1-1" and "R1-2" based on the step (a process known as subspacing). In this case, since "R1" and "R1-1" (and "R1-2") represent the same physical space, they have a "contains" relationship. As shown in the example, this can be represented as "ILC-R1-1" (and "ILC-R1-2").

```
{
  "id": "ILC-R1-1",
  "featureType": "InterLayerConnection",
  "connectedLayers": ["TL-Pedestrian", "TL-Wheelchair"],
  "connectedCells": ["R1", "R1-1"],
  "connectedNodes": ["N-R1", "N-R1-1"],
  "typeOfTopoExpression": "CONTAINS"
}
```

Listing 18 — An example of the InterLayerConnection object

REQUIREMENT 11: INTERLAYERCONNECTION SCHEMA	
IDENTIFIER	`/req/core/interlayerconnection`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req/core`
A	The **`InterLayerConnection`** object SHALL reference the connected **`ThematicLayer`** using the `connectedLayers` member.
B	The **`InterLayerConnection`** object SHALL use the `typeOfTopoExpression` member to represent the topological the values in `connectedLayers` (or, if that value is not null, `connectedNodes` or `connectedCells`).
C	The `typeOfTopoExpression` value SHALL be one of the following: "CONTAINS", "OVERLAPS", "EQUALS", "WITHIN", "CROSSES", or "OTHERS"
D	The `connectedLayers` value SHALL contain the same `id` as the linked **`ThematicLayer`** objects.
E	The `connectedNodes` value SHALL contain the same `id` as the linked **`Node`** objects.
F	The `connectedCells` value SHALL contain the same `id` as the linked **`CellSpace`** objects.

REQUIREMENT 11: INTERLAYERCONNECTION SCHEMA	
G	The values of the `connectedLayers`, `connectedNodes`, and `connectedCells` members SHALL be ordered and mapped to the same indices.
H	The `featuretype` value SHALL be “InterLayerConnection”.

7.11. ExternalReferenceType

The `externalReferenceType` object is used for the reference of an object to its corresponding object in an external data set. This is not a feature class of IndoorGML 2.0.

```
{
  "ExternalReferenceType": {
    "$anchor": "ExternalReferenceType",
    "type": "object",
    "properties": {
      "externalObject": {
        "type": "object",
        "properties": {
          "name": { "type": "string" },
          "uri": { "type": "string", "format": "uri" }
        },
        "required": ["name","uri"]
      },
      "informationSystem": { "type": "string", "format": "uri" }
    },
    "required": ["externalObject", "informationSystem"]
  }
}
```

Listing 19 — A JSON schema of the ExternalReferenceType

8

INDOORJSON NAVIGATION

8 INDOORJSON NAVIGATION

REQUIREMENTS CLASS 2	
IDENTIFIER	http://www.opengis.net/spec/indoorgml-2/2.0/json/req/navi
TARGET TYPE	Implementation Specification
CONFORMANCE CLASS	Conformance class A.2: http://www.opengis.net/spec/indoorgml-2/2.0/conf/navi
PREREQUISITES	http://www.opengis.net/spec/indoorgml/2.0/req http://www.opengis.net/spec/json-fg-1/1.0/req/core http://www.opengis.net/spec/json-fg-1/1.0/req/polyhedra
NORMATIVE STATEMENTS	Requirement 12: /req/navi/element Requirement 13: /req/navi/navigablespace Requirement 14: /req/navi/generalspace Requirement 15: /req/navi/transferspace Requirement 16: /req/navi/nonnavigablespace Requirement 17: /req/navi/objectspace Requirement 18: /req/navi/navigableboundary Requirement 19: /req/navi/nonnavigableboundary Requirement 20: /req/navi/route

REQUIREMENT 12: REQUIREMENT FOR INDOORJSON NAVIGATION EXTENSION	
IDENTIFIER	/req/navi/element
INCLUDED IN	Requirements class 2: http://www.opengis.net/spec/indoorgml-2/2.0/json/req/navi
A	IndoorGML Navigation Extension JSON elements implemented by a conforming instance document SHALL conform to the JSON schema in indoorjson.navi.schema.json.

8.1. NavigableSpace

NavigableSpace object is an instance of NavigableSpace, which represents a space where the user can move freely. The compartmentalized spaces such as corridor, door, lobby, hallway, big room are represented as **NavigableSpace**.

An **NavigableSpace** object shall contain:

- All members from **CellSpace**
- `locomotionType` is the allowed transportation mode in indoor space

```
{
  "NavigableSpace": {
    "$anchor": "NavigableSpace",
    "allOf": [
      {"$ref": "indoorjson.core.schema.json#/$defs/CellSpace"},
      {
        "type": "object",
        "properties": {
          "featureType": { "enum": ["NavigableSpace","GeneralSpace",
"TransferSpace"] },
          "locomotionType": {
            "anyOf": [
              { "enum": ["Flying","Rolling","Walking","Unspecified"] },
              { "type": "string", "minLength": 1 }
            ]
          }
        },
        "required": ["locomotionType"]
      }
    ]
  }
}
```

Listing 20 — A JSON schema of the NavigableSpace:

```
{
  "id": "NS1",
  "featureType": "NavigableSpace",
  "duality": "N1",
  "poi": false,
  "level": "1",
  "cellSpaceGeom": {...},
  "boundedBy": ["B1"],
  "locomotionType": "Walking"
}
```

Listing 21 — An example of the NavigableSpace object:

REQUIREMENT 13: NAVIGABLESPACE SCHEMA	
IDENTIFIER	`/req/navi/navigablespace`
INCLUDED IN	Requirements class 2: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req/navi`
A	A **NavigableSpace** object SHALL specify the transportation mode using the `locomotionType` member.
B	The `locomotionType` value SHALL be a user-defined code or one of the following: “Flying”, “Rolling”, “Walking”, or “Unspecified”

REQUIREMENT 13: NAVIGABLESPACE SCHEMA	
C	The `featuretype` value SHALL be “NavigableSpace”.

8.2. GeneralSpace

`GeneralSpace` object is an instance of GeneralSpace, which agents can use for a longer period of time and can serve as starting and target cell in navigation.

An **`GeneralSpace`** object shall contain:

- All members from **`NavigableSpace`**
- `function` specifies details about the function of the cell.

```
{
  "GeneralSpace": {
    "$anchor": "GeneralSpace",
    "allOf": [
      {"$ref": "#/$defs/NavigableSpace"},
      {
        "type": "object",
        "properties": {
          "featureType": { "enum": ["GeneralSpace"] },
          "function": {
            "anyOf": [
              { "enum": ["Administration","Business, trade","Education,
training","Recreation","Laboratory","Storage","Security"] },
              { "type": "string", "minLength": 1 }
            ]
          }
        },
        "required": ["function"]
      }
    ]
  }
}
```

Listing 22 — A JSON schema of the GeneralSpace:

```
{
  "id": "GS-R1",
  "featureType": "GeneralSpace",
  "duality": "N-R1",
  "poi": false,
  "level": "1",
  "cellSpaceGeom": {
    "geometry2D": {
      "type": "Polygon",
      "coordinates": [[[0,0.25], [3.25,0.25], [3.25,1.5], [0,1.5], [0,0.25]]]
    }
  },
  "boundedBy": ["B1", "B2", "B7", "B8"],
```

```
  "locomotionType": "Walking",
  "function": "Administration"
}
```

Listing 23 — An example of the GeneralSpace object:

REQUIREMENT 14: GENERALSPACE SCHEMA	
IDENTIFIER	`/req/navi/generalspace`
INCLUDED IN	Requirements class 2: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req/navi`
A	A **`GeneralSpace`** object SHALL specify the transportation mode using the `function` member.
B	The `function` value SHALL be a user-defined code or one of the following: “Administration”, ”Business, trade”,”Education, training”,”Recreation”,”Laboratory”,”Storage”, or “Security”
C	The `featuretype` value SHALL be “GeneralSpace”.

8.3. TransferSpace

`TransferSpace` object is an instance of GeneralSpace, which a space that provides passages between **`GeneralSpaces`**.

An **`TransferSpace`** object shall contain:

- All members from **`NavigableSpace`**
- `category` specifies opening categories (Door or Window).
- `function` describes whether the space is an “AnchorSpace” or a “ConnectionSpace”.

Note that an AnchorSpace is a space that allows for the connection between the indoor and the outdoor. A ConnectionSpace is a space connecting two indoor or two outdoor spaces.

```
{
  "TransferSpace": {
    "$anchor": "TransferSpace",
    "allOf": [
      {"$ref": "#/$defs/NavigableSpace"},
      {
        "type": "object",
        "properties": {
          "featureType": { "enum": ["TransferSpace"] },
          "category": {
            "anyOf": [
              { "enum": ["Door","Window"] },
              { "type": "string", "minLength": 1 }
            ]
```

```
          },
          "function": {
            "anyOf": [
              { "enum": ["AnchorSpace","ConnectionSpace"] },
              { "type": "string", "minLength": 1 }
            ]
          }
        },
        "required": ["category","function"]
      }
    ]
  }
}
```

Listing 24 — A JSON schema of the TransferSpace:

```
{
  "id": "TS-D1",
  "featureType": "TransferSpace",
  "poi": false,
  "cellSpaceGeom": {
    "geometry2D": {
      "type": "Polygon",
      "coordinates": [[[0.5,0.0], [1.0,0.0], [1.0,0.5], [0.5,1.0], [0.5,0.0]]]
    }
  },
  "category": "Door",
  "function": "ConnectionSpace"
}
```

Listing 25 — An example of the TransferSpace object:

REQUIREMENT 15: TRANSFERSPACE SCHEMA	
IDENTIFIER	`/req/navi/transferspace`
INCLUDED IN	Requirements class 2: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req/navi`
A	A **`TransferSpace`** object SHALL contain `category` and `function` members.
B	The `category` value SHALL be a user-defined code or one of the following: “Door” or “Window”
C	The `function` value SHALL be a user-defined code or one of the following: “AnchorSpace” for representing connection between the indoor and the outdoor or “ConnectionSpace” for representing connection between two indoor or outdoor spaces.
D	The `featuretype` value SHALL be “TransferSpace”.

8.4. NonNavigableSpace

NonNavigableSpace object is an instance of NonNavigableSpace, which **CellSpace** that are occupied by obstacles.

An **ObjectSpace** object shall contain:

- All members from **CellSpace**

```
{
  "NonNavigableSpace": {
    "$anchor": "NonNavigableSpace",
    "allOf": [
      {"$ref": "indoorjson.core.schema.json#/$defs/CellSpace"},
      {
        "type": "object",
        "properties": {
          "featureType": { "enum": ["NonNavigableSpace","ObjectSpace"] }
        }
      }
    ]
  }
}
```

Listing 26 — A JSON schema of the NonNavigableSpace:

REQUIREMENT 16: NONNAVIGABLESPACE SCHEMA	
IDENTIFIER	`/req/navi/nonnavigablespace`
INCLUDED IN	Requirements class 2: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req/navi`
A	The `featuretype` value SHALL be one of “NonNavigableSpace” or “ObjectSpace”.

8.5. ObjectSpace

ObjectSpace object is an instance of ObjectSpace, when **NonNavigableSpace** contains some objects that makes it non-navigable.

An **ObjectSpace** object shall contain:

- All members from **NonNavigableSpace**
- optionally, `containedFeatures` describes the number of objects encapsulated within the **ObjectSpace**.

- optionally, `description` describes any relevant information regarding the objects contained within the space in plain text.

```
{
  "ObjectSpace": {
    "$anchor": "ObjectSpace",
    "allOf": [
      {"$ref": "#/$defs/NonNavigableSpace"},
      {
        "type": "object",
        "properties": {
          "featureType": { "enum": ["ObjectSpace"] },
          "containedFeatures": { "type": "integer"},
          "description": { "type": "string"}
        }
      }
    ]
  }
}
```

Listing 27 — A JSON schema of the ObjectSpace:

```
{
  "id": "OS1",
  "featureType": "ObjectSpace",
  "poi": true,
  "cellSpaceGeom": {...},
  "containedFeatures": 1,
  "description": "vending machine"
}
```

Listing 28 — An example of the ObjectSpace object:

REQUIREMENT 17: OBJECTSPACE SCHEMA	
IDENTIFIER	`/req/navi/objectspace`
INCLUDED IN	Requirements class 2: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req/navi`
A	The `featuretype` value SHALL be “ObjectSpace”.

8.6. NavigableBoundary

`NavigableBoundary` object is an instance of NavigableBoundary, which provides further information related to **`NavigableSpace`**.

An **`NavigableBoundary`** object shall contain:

- All members from **`CellBoundary`**

- `navigableBoundaryFunction` describes whether the boundary is an “AnchorBoundary” or a “ConnectionBoundary”.
- optionally, `boundaryOrientation` is a flag that specifies whether to use the geometry represented by `cellBoundaryGeom` as-is (’true’) or in reverse (’false’).

Note that an AnchorBoundary is the boundary of AnchorSpace that enables the connection between indoor and outdoor spaces. A ConnectionBoundary is the boundary of a ConnectionSpace that connects two indoor spaces or two outdoor spaces.

```
{
  "NavigableBoundary": {
    "$anchor": "NavigableBoundary",
    "allOf": [
      {"$ref": "indoorjson.core.schema.json#/$defs/CellBoundary"},
      {
        "type": "object",
        "properties": {
          "featureType": { "enum": ["NavigableBoundary"] },
          "boundaryOrientation": { "type": "boolean" },
          "navigableBoundaryFunction": {
            "anyOf": [
              { "enum": ["AnchorBoundary","ConnectionBoundary"] },
              { "type": "string", "minLength": 1 }
            ]
          }
        },
        "required": ["navigableBoundaryFunction"]
      }
    ]
  }
}
```

Listing 29 — A JSON schema of the NavigableBoundary:

```
{
  "id": "NB-B1",
  "featureType": "NavigableBoundary",
  "isVirtual": false,
  "duality": "E1",
  "cellBoundaryGeom": {
    "geometry1D": {
      "type": "LineString",
      "coordinates": [[0.5,0.25],[1.0,0.25]]
    }
  },
  "boundaryOrientation": false,
  "navigableBoundaryFunction": "ConnectionBoundary"
}
```

Listing 30 — An example of the NavigableBoundary object:

REQUIREMENT 18: NAVIGABLEBOUNDARY SCHEMA

IDENTIFIER	`/req/navi/navigableboundary`

REQUIREMENT 18: NAVIGABLEBOUNDARY SCHEMA	
INCLUDED IN	Requirements class 2: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req/navi`
A	A **`NavigableBoundary`** object SHALL specify the function type using the `navigableBoundaryFunction` member.
B	The `navigableBoundaryFunction` value SHALL be a user-defined code or one of the following: “AnchorBoundary” or “ConnectionBoundary”
C	The `featuretype` value SHALL be “NavigableBoundary”.

8.7. NonNavigableBoundary

`NonNavigableBoundary` object is an instance of NonNavigableBoundary, which represent the boundaries between two **`NonNavigableSpace`** cells or between a **`NavigableSpace`** and a **`NonNavigableSpace`** cells.

An **`NonNavigableBoundary`** object shall contain:

- All members from **`CellBoundary`**

```
{
  "NavigableBoundary": {
    "$anchor": "NonNavigableBoundary",
    "allOf": [
      {"$ref": "indoorjson.core.schema.json#/$defs/CellBoundary"},
      {
        "type": "object",
        "properties": {
          "featureType": { "enum": ["NonNavigableBoundary"] },
        }
      }
    ]
  }
}
```

Listing 31 — A JSON schema of the NonNavigableBoundary:

REQUIREMENT 19: NONNAVIGABLEBOUNDARY SCHEMA	
IDENTIFIER	`/req/navi/nonnavigableboundary`
INCLUDED IN	Requirements class 2: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req/navi`
A	The `featuretype` value SHALL be “NonNavigableBoundary”.

8.8. Route

Route object is an instance of Route, which represents a subset of network from **DualSpace**, to describe a path for navigation within an indoor space.

An **Route** object shall contain:

- routeNode is ordered sequences of **Node** references to describe the different parts of the route path.
- routeEdge is ordered sequences of **Edge** references to describe the different parts of the route path.
- optionally, creationDate indicates when a specific route was created.

```
{
  "Route": {
    "$anchor": "Route",
    "type": "object",
    "properties": {
      "creationDate": { "type": "string", "format": "date-time"},
      "routeNode": {
        "type": "array",
        "minItems": 2,
        "items": {
          "anyOf": [
            {"$ref": "indoorjson.core.schema.json#/$defs/Node"},
            {"type": "string", "format": "uri-reference"}
          ]
        }
      },
      "routeEdge": {
        "type": "array",
        "minItems": 1,
        "items": {
          "anyOf": [
            {"$ref": "indoorjson.core.schema.json#/$defs/Edge"},
            {"type": "string", "format": "uri-reference"}
          ]
        }
      }
    },
    "required": ["routeNode","routeEdge"]
  }
}
```

Listing 32 — A JSON schema of the Route:

```
{
  "routeNode": ["N-D1","N-R1"],
  "routeEdge": ["E1"]
}
```

Listing 33 — An example of the Route object (reference ID only):

```
{
  "routeNode": [
```

```
    {
      "id": "N-D1",
      "featureType": "Node",
      "duality": "D1",
      "geometry": {
        "type": "Point",
        "coordinates": [0.75,0.125]
      },
      "connects": ["E1"]
    },
    {
      "id": "N-R1",
      "featureType": "Node",
      "duality": "R1",
      "geometry": {
        "type": "Point",
        "coordinates": [1.625,0.875]
      },
      "connects": ["E1","E2","E7","E8"]
    }
  ],
  "routeEdge": [
    {
      "id": "E1",
      "featureType": "Edge",
      "duality": "B1",
      "weight": 1.0,
      "geometry": {
        "type": "LineString",
        "coordinates": [[0.75,0.125],[0.75,0.875],[1.625,0.875]]
      },
      "connects": ["N-D1", "N-R1"]
    }
  ]
}
```


Listing 34 — An example of the Route object (using Node (and Edge) object itself):

REQUIREMENT 20: ROUTE SCHEMA	
IDENTIFIER	`/req/navi/route`
INCLUDED IN	Requirements class 2: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req/navi`
A	A **`Route`** object SHALL contain `routeNode` and `routeEdge` members.
B	The `routeNode` value SHALL contain either the `id` of the linked **`Node`** object or the **`Node`** object itself.
C	The `routeEdge` value SHALL contain either the same `id` as the linked **`Edge`** objects or the **`Edge`** object itself.
D	The `routeNode` and `routeEdge` SHALL be an ordered list.
E	An **`Edge`** object whose `connected` value contains two consecutive IDs within `routeNode` SHALL be included in `routeEdge` value.

REQUIREMENT 20: ROUTE SCHEMA	
F	The `connected` value of a **Edge** object corresponding to two consecutive IDs in `routeEdge` SHALL be contained the ID of the same **Node** object.

A

ANNEX A (NORMATIVE) CONFORMANCE CLASS ABSTRACT TEST SUITE

A ANNEX A (NORMATIVE) CONFORMANCE CLASS ABSTRACT TEST SUITE

A.1. Conformance Class Core

CONFORMANCE CLASS A.1	
IDENTIFIER	`http://www.opengis.net/spec/indoorgml-2/2.0/conf/core`
REQUIREMENTS CLASS	Requirements class 1: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req/core`
TARGET TYPE	Implementation Specification
CONFORMANCE TESTS	Abstract test A.1: `/conf/core/element` Abstract test A.2: `/conf/core/indoorjson` Abstract test A.3: `/conf/core/indoorfeature` Abstract test A.4: `/conf/core/thematiclayer` Abstract test A.5: `/conf/core/primalspacelayer` Abstract test A.6: `/conf/core/cellspace` Abstract test A.7: `/conf/core/cellboundary` Abstract test A.8: `/conf/core/dualspacelayer` Abstract test A.9: `/conf/core/node` Abstract test A.10: `/conf/core/edge` Abstract test A.11: `/conf/core/interlayerconnection`

ABSTRACT TEST A.1	
IDENTIFIER	`/conf/core/element`
REQUIREMENT	Requirement 2: `/req/core/element`

ABSTRACT TEST A.1

TEST PURPOSE	Verify that instance documents using the IndoorGML Core JSON elements validate against the JSON schema specified in indoorjson.core.schema.json.
TEST METHOD	Validate the instance document against the indoorjson.core.schema.json schema to verify the above requirement. The process may be using an appropriate software tool for validation or be a manual process that checks all definitions from the JSON schema specification.

ABSTRACT TEST A.2

IDENTIFIER	`/conf/core/indoorjson`
REQUIREMENT	Requirement 1: `/req/core/indoorjson`
TEST PURPOSE	For each IndoorJSON object verify that if the IndoorJSON object (and inherited objects) is contained `id` and `featuretype` members.
TEST METHOD	Inspect the instance document to verify the above requirement.

ABSTRACT TEST A.3

IDENTIFIER	`/conf/core/indoorfeature`
REQUIREMENT	Requirement 3: `/req/core/indoorfeature`
TEST PURPOSE	For the following members of **IndoorFeatures** object, verify that: • The **IndoorFeatures** object is contained at least one **ThematicLayer** object as item of `layers` value. • The `featuretype` value is "IndoorFeatures". • If the `layerConnections` value is not null, it contains the set of **InterLayerConnection** objects.
TEST METHOD	Inspect the instance document to verify the above requirement.

ABSTRACT TEST A.4

IDENTIFIER	`/conf/core/thematiclayer`
REQUIREMENT	Requirement 4: `/req/core/thematiclayer`
TEST PURPOSE	For the following members of **ThematicLayer** object, verify that:

ABSTRACT TEST A.4

	• The `semanticExtension` value (boolean) is declared based on whether there is an extension module that contains additional semantic information. • The `theme` value is one of the following: “Virtual”, “Physical”, “Tags”, or “Unknown” • The `featuretype` value is “ThematicLayer” • If the `primalSpcae` value is not null, it contains the **PrimalSpaceLayer** object. • If the `dualSpace` value is not null, it contains the **DualSpace** object.
TEST METHOD	Inspect the instance document to verify the above requirement.

ABSTRACT TEST A.5

IDENTIFIER	`/conf/core/primalspacelayer`
REQUIREMENT	Requirement 5: `/req/core/primalspacelayer`
TEST PURPOSE	For the following members of **PrimalSpaceLayer** object, verify that: • The **PrimalSpaceLayer** object is contained at least one **CellSpace** object as item of `cellSpaceMember` value. • The geometries of each **CellSpace** object contained in `cellSpaceMember` do not intersect with one another. • The geometries of each **CellBoundary** object contained in `cellBounaryMember` do not intersect with one another. • The `featuretype` value is “PrimalSpaceLayer”.
TEST METHOD	Inspect the instance document to verify the above requirement.

ABSTRACT TEST A.6

IDENTIFIER	`/conf/core/cellspace`
REQUIREMENT	Requirement 6: `/req/core/cellspace`
TEST PURPOSE	For the following members of **CellSpace** object, verify that: • The `poi` value (boolean) is declared based on whether it is a point of interest space. • If the `duality` value is not null, it contains the `id` value of the **Node** object in the same instance document. • If the `boundedBy` value is not null, it contains the `id` value of the **CellBoundary** object in the same instance document. • The geometries of each **CellBoundary** object contained in `boundedBy` value do not exceed the extent of the `cellSpaceGeom` value.

ABSTRACT TEST A.6

	• The `featuretype` value is one of the following: “CellSpace”, “NonNavigableSpace”, or “ObjectSpace”.
TEST METHOD	Inspect the instance document to verify the above requirement.

ABSTRACT TEST A.7

IDENTIFIER	`/conf/core/cellboundary`
REQUIREMENT	Requirement 7: `/req/core/cellboundary`
TEST PURPOSE	For the following members of **CellBoundary** object, verify that: • The `isVirtual` value (boolean) is declared based on whether it is virtual boundary. • If the `duality` value is not null, it contains the `id` value of the **Edge** object in the same instance document. • The `featuretype` value is one of the following: “CellBoundary”, or “NonNavigable Boundary”.
TEST METHOD	Inspect the instance document to verify the above requirement.

ABSTRACT TEST A.8

IDENTIFIER	`/conf/core/dualspacelayer`
REQUIREMENT	Requirement 8: `/req/core/dualspacelayer`
TEST PURPOSE	For the following members of **DualSpaceLayer** object, verify that: • The **DualSpaceLayer** object is contained at least one **Node** object as item of `nodeMember` value. • The `isLogical` value (boolean) is declared based on whether the provided graph is a geometric or a logical. • If the `isLogical` value is False, the geometry of each **Node** object contained in `nodeMember` is spatially located within their corresponding **CellSpace**. • The `isDirected` value (boolean) is declared based on the provided graph is directed. • If the `edgeMember` value is not null, it contains the **Edge** object. • The `featuretype` value is “DualSpaceLayer”.
TEST METHOD	Inspect the instance document to verify the above requirement.

ABSTRACT TEST A.9

IDENTIFIER	`/conf/core/node`
REQUIREMENT	Requirement 9: `/req/core/node`
TEST PURPOSE	For the following members of **Node** object, verify that: • The `duality` value contains the `id` value of the **CellSpace** object in the same instance document. • If the `connects` value is not null, it contains the `id` value of the **Edge** object in the same instance document. • The `featuretype` value is "Node".
TEST METHOD	Inspect the instance document to verify the above requirement.

ABSTRACT TEST A.10

IDENTIFIER	`/conf/core/edge`
REQUIREMENT	Requirement 10: `/req/core/edge`
TEST PURPOSE	For the following members of **Edge** object, verify that: • The `connects` value contains two `id` values (start and end) of the **Node** object in the same instance document. • If the parent **DualSpace** object's `isDirected` value is True, the `connects` value interpreted as ordered sequence (start to end). • If the `duality` value is not null, it contains the `id` value of the **CellBoundary** object in the same instance document. • The start and end coordinates of the **Edge** geometry and geometry of the **Nodes** included in `connects` is the same. • The `featuretype` value is "Edge".
TEST METHOD	Inspect the instance document to verify the above requirement.

ABSTRACT TEST A.11

IDENTIFIER	`/conf/core/interlayerconnection`
REQUIREMENT	Requirement 11: `/req/core/interlayerconnection`
TEST PURPOSE	For the following members of **InterLayerConnection** object, verify that: • The `connectedLayers` value contains two `id` values of the **ThematicLayer** object in the same instance document.

ABSTRACT TEST A.11	
	• If the value of `connectedNodes` is not null, it contains the `id` values of **Node** objects in the same instance document, and each **Node** object is contained within a **ThematicLayer** object specified in `connectedLayers`. In this case, the values of `connectedLayers` and `connectedNodes` are matched in order. • If the value of `connectedCells` is not null, it contains the `id` values of **CellSpace** objects in the same instance document, and each **CellSpace** object is contained within a **ThematicLayer** object specified in `connectedLayers`. In this case, the values of `connectedLayers` and `connectedNodes` are matched in order. • The `typeOfTopoExpression` value is one of the following: "CONTAINS", "OVERLAPS", "EQUALS", "WITHIN", "CROSSES", or "OTHERS" • The `featuretype` value is "InterLayerConnection".
TEST METHOD	Inspect the instance document to verify the above requirement.

A.2. Conformance Class Navigation Extension

CONFORMANCE CLASS A.2	
IDENTIFIER	`http://www.opengis.net/spec/indoorgml-2/2.0/conf/navi`
REQUIREMENTS CLASS	Requirements class 2: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req/navi`
TARGET TYPE	Implementation Specification
CONFORMANCE TESTS	Abstract test A.12: `/conf/navi/element` Abstract test A.13: `/conf/navi/navigablespace` Abstract test A.14: `/conf/navi/generalspace` Abstract test A.15: `/conf/navi/transferspace` Abstract test A.16: `/conf/navi/nonnavigablespace` Abstract test A.17: `/conf/navi/objectspace` Abstract test A.18: `/conf/navi/navigableboundary` Abstract test A.19: `/conf/navi/nonnavigableboundary` Abstract test A.20: `/conf/navi/route`

ABSTRACT TEST A.12	
IDENTIFIER	`/conf/navi/element`
REQUIREMENT	Requirement 12: `/req/navi/element`

ABSTRACT TEST A.12

TEST PURPOSE	Verify that instance documents using the IndoorGML Core JSON elements validate against the JSON schema specified in indoorjson.navi.schema.json.
TEST METHOD	Validate the instance document against the indoorjson.navi.schema.json schema to verify the above requirement. The process may be using an appropriate software tool for validation or be a manual process that checks all definitions from the JSON schema specification.

ABSTRACT TEST A.13

IDENTIFIER	`/conf/navi/navigablespace`
REQUIREMENT	Requirement 13: `/req/navi/navigablespace`
TEST PURPOSE	For the following members of **NavigableSpace** object, verify that: • Check the validation criteria for "/conf/core/cellspace" based on the inheritance relationship, excluding `featuretype`. • The `locomotionType` value is one of the following: "Flying", "Rolling", "Walking", or "Unspecified" • The `featuretype` value is "NavigableSpace".
TEST METHOD	Inspect the instance document to verify the above requirement.

ABSTRACT TEST A.14

IDENTIFIER	`/conf/navi/generalspace`
REQUIREMENT	Requirement 14: `/req/navi/generalspace`
TEST PURPOSE	For the following members of **GeneralSpace** object, verify that: • Check the validation criteria for "/conf/navi/navigablespace" based on the inheritance relationship, excluding `featuretype`. • The `function` value is one of the following: "Administration","Business, trade", "Education, training","Recreation","Laboratory","Storage", or "Security" • The `featuretype` value is "GeneralSpace".
TEST METHOD	Inspect the instance document to verify the above requirement.

ABSTRACT TEST A.15

IDENTIFIER	`/conf/navi/transferspace`

ABSTRACT TEST A.15

REQUIREMENT	Requirement 15: `/req/navi/transferspace`
TEST PURPOSE	For the following members of **TransferSpace** object, verify that: • Check the validation criteria for "/conf/navi/navigablespace" based on the inheritance relationship, excluding `featuretype`. • The `category` value is one of the following: "Door" or "Window" • The `function` value is one of the following: "AnchorSpace" or "ConnectionSpace" • The `featuretype` value is "TransferSpace".
TEST METHOD	Inspect the instance document to verify the above requirement.

ABSTRACT TEST A.16

IDENTIFIER	`/conf/navi/nonnavigablespace`
REQUIREMENT	Requirement 16: `/req/navi/nonnavigablespace`
TEST PURPOSE	For the following members of **NonNavigableSpace** object, verify that: • Check the validation criteria for "/conf/core/cellspace" based on the inheritance relationship, excluding `featuretype`. • The `featuretype` value is one of the following: "NonNavigableSpace" or "ObjectSpace".
TEST METHOD	Inspect the instance document to verify the above requirement.

ABSTRACT TEST A.17

IDENTIFIER	`/conf/navi/objectspace`
REQUIREMENT	Requirement 17: `/req/navi/objectspace`
TEST PURPOSE	For the following members of **NonNavigableSpace** object, verify that: • Check the validation criteria for "/conf/navi/nonnavigablespace" based on the inheritance relationship, excluding `featuretype`. • The `featuretype` value is "ObjectSpace".
TEST METHOD	Inspect the instance document to verify the above requirement.

ABSTRACT TEST A.18

IDENTIFIER	`/conf/navi/navigableboundary`
REQUIREMENT	Requirement 18: `/req/navi/navigableboundary`
TEST PURPOSE	For the following members of **NavigableSpace** object, verify that: • Check the validation criteria for "/conf/core/cellboundary" based on the inheritance relationship, excluding `featuretype`. • The `navigableBoundaryFunction` value is one of the following: "AnchorBoundary" or "ConnectionBoundary" • The `featuretype` value is "NavigableBoundary".
TEST METHOD	Inspect the instance document to verify the above requirement.

ABSTRACT TEST A.19

IDENTIFIER	`/conf/navi/nonnavigableboundary`
REQUIREMENT	Requirement 19: `/req/navi/nonnavigableboundary`
TEST PURPOSE	For the following members of **NonNavigableSpace** object, verify that: • Check the validation criteria for "/conf/core/cellboundary" based on the inheritance relationship, excluding `featuretype`. • The `featuretype` value is "NonNavigableBoundary".
TEST METHOD	Inspect the instance document to verify the above requirement.

ABSTRACT TEST A.20

IDENTIFIER	`/conf/navi/route`
REQUIREMENT	Requirement 20: `/req/navi/route`
TEST PURPOSE	For the following members of **Route** object, verify that: • The `routeNode` contains the `id` values of **Node** objects in the same instance document, and each **Node** object is contained within the same **DualSpace** object. • The `routeEdge` contains the `id` values of **Edge** objects in the same instance document, and each **Edge** object is contained within the same **DualSpace** object. • The **Edge** objects whose `connected` values contain two consecutive IDs within `routeNode` are included in the `routeEdge` value. • The `connected` values of **Edge** objects corresponding to two consecutive IDs within `routeEdge` contain the ID of the same **Node** object.

ABSTRACT TEST A.20

TEST METHOD Inspect the instance document to verify the above requirement.

B

ANNEX B (NORMATIVE) JSON SCHEMAS

B ANNEX B (NORMATIVE) JSON SCHEMAS

B.1. JSON Schema of a IndoorJSON Core

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://www.opengis.net/spec/indoorgml/2.0/json/indoorjson.core.schema.
json",
  "title": "IndoorJSON for IndoorGML 2.0 core part",
  "description": "IndoorJSON Core",
  "oneOf": [
    {"$ref": "#/$defs/IndoorFeatures"       },
    {"$ref": "#/$defs/ThematicLayer"        },
    {"$ref": "#/$defs/InterLayerConnection" },
    {"$ref": "#/$defs/PrimalSpaceLayer"     },
    {"$ref": "#/$defs/CellSpace"            },
    {"$ref": "#/$defs/CellBoundary"         },
    {"$ref": "#/$defs/DualSpaceLayer"       },
    {"$ref": "#/$defs/Node"                 },
    {"$ref": "#/$defs/Edge"                 }
  ],
  "$defs": {

    "IndoorFeatures": {
      "$anchor": "IndoorFeatures",
      "type": "object",
      "properties": {
        "id": { "type": "string" },
        "featureType": { "enum": ["IndoorFeatures"] },
        "layers": {
          "type": "array",
          "minItems": 1,
          "items": { "$ref": "#/$defs/ThematicLayer" },
          "uniqueItems": true
        },
        "layerConnections": {
          "type": "array",
          "items": { "$ref": "#/$defs/InterLayerConnection" },
          "uniqueItems": true
        }
      },
      "required": ["id","featureType","layers"]
    },

    "ThematicLayer": {
      "$anchor": "ThematicLayer",
```

```
      "type": "object",
      "properties": {
        "id": { "type": "string" },
        "featureType": { "enum": ["ThematicLayer"] },
        "semanticExtension": { "type": "boolean" },
        "theme": { "enum": ["Virtual", "Physical", "Tags", "Unknown"] },
        "primalSpace": { "$ref": "#/$defs/PrimalSpaceLayer" },
        "dualSpace": { "$ref": "#/$defs/DualSpaceLayer" }
      },
      "required": ["id","featureType","semanticExtension","theme"]
    },

    "PrimalSpaceLayer": {
      "$anchor": "PrimalSpaceLayer",
      "type": "object",
      "properties": {
        "id": { "type": "string" },
        "featureType": { "enum": ["PrimalSpaceLayer"] },
        "creationDatetime": { "type": "string", "format": "date-time" },
        "terminationDatetime": { "type": "string", "format": "date-time" },
        "cellSpaceMember": {
          "type": "array",
          "minItems": 1,
          "items": { "$ref": "#/$defs/CellSpace" },
          "uniqueItems": true
        },
        "cellBoundaryMember": {
          "type": "array",
          "items": { "$ref": "#/$defs/CellBoundary" },
          "uniqueItems": true
        }
      },
      "required": ["id","featureType","cellSpaceMember"]
    },

    "CellSpace": {
      "$anchor": "CellSpace",
      "type": "object",
      "properties": {
        "id": { "type": "string" },
        "featureType": { "enum": ["CellSpace",
          "NavigableSpace","GeneralSpace","TransferSpace",
          "NonNavigableSpace","ObjectSpace"] },
        "cellSpaceName:": { "type": "string" },
        "level": { "type": "string" },
        "poi": { "type":  "boolean" },
        "duality" : { "type": "string", "format": "uri-reference" },
        "cellSpaceGeom" : {
          "type": "object",
          "properties": {
            "geometry2D": { "$ref": "https://schemas.opengis.net/json-fg/
geometry-object.json#/$defs/Polygon" },
            "geometry3D": { "$ref": "https://schemas.opengis.net/json-fg/
geometry-object.json#/$defs/Polyhedron" }
          }
        },
        "boundedBy": {
          "type": "array",
          "items": { "type": "string", "format": "uri-reference" },
          "uniqueItems": true
        },
        "externalReference": { "$ref": "#/$defs/ExternalReferenceType" }
      },
```

```
      "required": ["id","featureType","poi"]
    },

    "CellBoundary": {
      "$anchor": "CellBoundary",
      "type": "object",
      "properties": {
        "id": { "type": "string" },
        "featureType": { "enum": ["CellBoundary","NavigableBoundary",
"NonNavigableBoundary"] },
        "isVirtual": { "type":  "boolean" },
        "duality" : { "type": "string", "format": "uri-reference" },
        "cellBoundaryGeom" : {
          "type": "object",
          "properties": {
            "geometry1D": { "$ref": "https://schemas.opengis.net/json-fg/
geometry-object.json#/$defs/LineString" },
            "geometry2D": { "$ref": "https://schemas.opengis.net/json-fg/
geometry-object.json#/$defs/Polygon" }
          }
        },
        "externalReference": { "$ref": "#/$defs/ExternalReferenceType" }
      },
      "required": ["id","featureType","isVirtual"]
    },

    "DualSpaceLayer": {
      "$anchor": "DualSpaceLayer",
      "type": "object",
      "properties": {
        "id": { "type": "string" },
        "featureType": { "enum": ["DualSpaceLayer"] },
        "creationDatetime": { "type": "string", "format": "date-time" },
        "terminationDatetime": { "type": "string", "format": "date-time" },
        "isLogical": { "type": "boolean" },
        "isDirected": { "type": "boolean" },
        "nodeMember": {
          "type": "array",
          "minItems": 1,
          "items": { "$ref": "#/$defs/Node" },
          "uniqueItems": true
        },
        "edgeMember": {
          "type": "array",
          "items": { "$ref": "#/$defs/Edge" },
          "uniqueItems": true
        }
      },
      "required": ["id","featureType","isLogical","isDirected","nodeMember"]
    },

    "Node": {
      "$anchor": "Node",
      "type": "object",
      "properties": {
        "id": { "type": "string" },
        "featureType": { "enum": ["Node"] },
        "duality" : { "type": "string", "format": "uri-reference" },
        "geometry" : { "$ref": "https://schemas.opengis.net/json-fg/geometry-
object.json#/$defs/Point" },
        "connects" : {
          "type": "array",
          "items": { "type": "string", "format": "uri-reference" },
```

```
          "uniqueItems": true
        }
      },
      "required": ["id","featureType","duality"]
    },

    "Edge": {
      "$anchor": "Edge",
      "type": "object",
      "properties": {
        "id": { "type": "string" },
        "featureType": { "enum": ["Edge"] },
        "duality" : { "type": "string", "format": "uri-reference" },
        "weight" : { "type": "number" },
        "geometry" : { "$ref": "https://schemas.opengis.net/json-fg/geometry-
object.json#/$defs/LineString" },
        "connects" : {
          "type": "array",
          "items": { "type": "string", "format": "uri-reference" },
          "minItems": 2,
          "maximum": 2,
          "uniqueItems": true
        }
      },
      "required": ["id","featureType","weight","connects"]
    },

    "InterLayerConnection": {
      "$anchor": "InterLayerConnection",
      "type": "object",
      "properties": {
        "id": { "type": "string" },
        "featureType": { "enum": ["InterLayerConnection"] },
        "connectedLayers": {
          "type": "array",
          "items": { "type": "string", "format": "uri-reference" },
          "minItems": 2,
          "maximum": 2,
          "uniqueItems": true
        },
        "connectedNodes" : {
          "type": "array",
          "items": { "type": "string", "format": "uri-reference" },
          "minItems": 2,
          "maximum": 2,
          "uniqueItems": true
        },
        "connectedCells" : {
          "type": "array",
          "items": { "type": "string", "format": "uri-reference" },
          "minItems": 2,
          "maximum": 2,
          "uniqueItems": true
        },
        "typeOfTopoExpression" : {
          "enum": ["CONTAINS", "OVERLAPS", "EQUALS", "WITHIN", "CROSSES",
"OTHERS"]
        },
        "comment" : { "type": "string" }
      },
      "required": ["id","featureType","connectedLayers","typeOfTopoExpression"]
    },
```

```
    "ExternalReferenceType": {
      "$anchor": "ExternalReferenceType",
      "type": "object",
      "properties": {
        "externalObject": {
          "type": "object",
          "properties": {
            "name": { "type": "string" },
            "uri": { "type": "string", "format": "uri" }
          },
          "required": ["name","uri"]
        },
        "informationSystem": { "type": "string", "format": "uri" }
      },
      "required": ["externalObject", "informationSystem"]
    }

  }
}
```

Listing B.1

B.2. JSON Schema of a IndoorJSON Navigation

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://www.opengis.net/spec/indoorgml/2.0/json/indoorjson.navi.schema.
json",
  "title": "IndoorJSON for IndoorGML 2.0 navigation part",
  "description": "IndoorJSON Navigation",
  "oneOf": [
    {"$ref": "#/$defs/NavigableSpace"       },
    {"$ref": "#/$defs/NonNavigableSpace"    },
    {"$ref": "#/$defs/GeneralSpace"         },
    {"$ref": "#/$defs/TransferSpace"        },
    {"$ref": "#/$defs/ObjectSpace"          },
    {"$ref": "#/$defs/NavigableBoundary"    },
    {"$ref": "#/$defs/NonNavigableBoundary" },
    {"$ref": "#/$defs/Route"                }
  ],
  "$defs": {

    "NavigableSpace": {
      "$anchor": "NavigableSpace",
      "allOf": [
        {"$ref": "indoorjson.core.schema.json#/$defs/CellSpace"},
        {
          "type": "object",
          "properties": {
            "locomotionType": { "enum": ["Flying","Rolling","Walking",
"Unspecified"]}
          },
          "required": ["locomotionType"]
        }
      ]
    },

    "NonNavigableSpace": {
```

```
      "$anchor": "NonNavigableSpace",
      "allOf": [
        {"$ref": "indoorjson.core.schema.json#/$defs/CellSpace"}
      ]
    },

    "GeneralSpace": {
      "$anchor": "GeneralSpace",
      "allOf": [
        {"$ref": "#/$defs/NavigableSpace"},
        {
          "type": "object",
          "properties": {
            "function": {
              "anyOf": [
                { "enum": ["Administration","Business, trade","Education,
training","Recreation","Laboratory","Storage","Security"] },
                { "type": "string", "minLength": 1 }
              ]
            }
          },
          "required": ["function"]
        }
      ]
    },

    "TransferSpace": {
      "$anchor": "TransferSpace",
      "allOf": [
        {"$ref": "#/$defs/NavigableSpace"},
        {
          "type": "object",
          "properties": {
            "category": {
              "anyOf": [
                { "enum": ["Door","Window"] },
                { "type": "string", "minLength": 1 }
              ]
            },
            "function": {
              "anyOf": [
                { "enum": ["AnchorSpace","ConnectionSpace"] },
                { "type": "string", "minLength": 1 }
              ]
            }
          },
          "required": ["category","function"]
        }
      ]
    },

    "ObjectSpace": {
      "$anchor": "ObjectSpace",
      "allOf": [
        {"$ref": "#/$defs/NonNavigableSpace"},
        {
          "type": "object",
          "properties": {
            "containedFeatures": { "type": "integer"},
            "description": { "type": "string"}
          }
        }
      ]
```

```
    },

    "NavigableBoundary": {
      "$anchor": "NavigableBoundary",
      "allOf": [
        {"$ref": "indoorjson.core.schema.json#/$defs/CellBoundary"},
        {
          "type": "object",
          "properties": {
            "boundaryOrientation": { "type": "boolean"},
            "navigableBoundaryFunction": { "enum": ["AnchorBoundary",
"ConnectionBoundary"]}
          },
          "required": ["navigableBoundaryFunction"]
        }
      ]
    },

    "NonNavigableBoundary": {
      "$anchor": "NonNavigableBoundary",
      "allOf": [
        {"$ref": "indoorjson.core.schema.json#/$defs/CellBoundary"}
      ]
    },

    "Route": {
      "$anchor": "Route",
      "type": "object",
      "properties": {
        "creationDate": { "type": "string", "format": "date-time"},
        "routeNode": {
          "type": "array",
          "minItems": 2,
          "items": {"$ref": "indoorjson.core.schema.json#/$defs/Node"}
        },
        "routeEdge": {
          "type": "array",
          "minItems": 1,
          "items": {"$ref": "indoorjson.core.schema.json#/$defs/Edge"}
        }
      },
      "required": ["routeNode","routeEdge"]
    }
  }
}
```

Listing B.2

C

ANNEX C (INFORMATIVE) EXAMPLE OF CODELIST

ANNEX C (INFORMATIVE) EXAMPLE OF CODELIST

For mapping indoor space features to IndoorNavigation module classes, space features have to be classified by functions and usages of space. Because misspellings or different names for the same notion brings the problems in data interoperability, in order to overcome the typo errors, an example of CodeList is provided in this annex to specify the attribute values including space category types and space function types. The CodeLists are defined from OmniClass (Table 13 and Table 14) created and used by the North American architectural, engineering and construction (AEC) industry.

C.1. GeneralSpace

C.1.1. GeneralSpaceFunctionType

Code list derived from OmniClass Table 13:

- T1000 — Corridor
- T1010 — Breezeway
- T1020 — Box Lobby
- T1030 — Elevator Lobby
- T1040 — Landing
- T1050 — Aisle
- T1060 — Ramp
- T1070 — Concourse
- T1080 — Moving walkway
- T1090 — Entry Lobby
- T1100 — Jet way

- T1110 — Elevator Shaft
- T1120 — Stair
- T1130 — Chute
- G1000 — Elevator Machine Room
- G1010 — Fire Command Center
- G1020 — Men's Restroom
- G1030 — Unisex Restroom
- G1040 — Refrigerant Machinery Room
- G1050 — Incinerator Room
- G1060 — Gas Room
- G1070 — Liquid Use, Dispensing and Mixing Room
- G1080 — Electrical Room
- G1090 — Telecommunications Room
- G1100 — Hazardous Waste Storage
- G1110 — Building Manager Office
- G1120 — Guard Stations
- G1130 — Women's Restroom
- G1140 — Furnace Room
- G1150 — Fuel Room
- G1160 — Liquid Storage Room
- G1170 — Hydrogen Cutoff Room
- G1180 — Switch Room
- G1190 — Classrooms
- G1200 — Assembly Hall
- G1210 — Physics Teaching Laboratory
- G1220 — Open Class Laboratory
- G1230 — Laboratory Service Space
- G1240 — Computer Lab

- G1250 — Training Support Space
- G1260 — Study Room
- G1270 — Evidence Room
- G1280 — Witness Stand
- G1290 — Robing Area
- G1300 — Council Chambers
- G1310 — Armory
- G1320 — General Performance Spaces
- G1330 — Orchestra Pit
- G1340 — Performance Hall
- G1350 — Audience Space
- G1360 — Audience Seating Space
- G1370 — Projection Booth
- G1380 — Stage Wings
- G1390 — Art Gallery
- G1400 — Sculpture Garden
- G1410 — Recording Studio
- G1420 — Photo Lab
- G1430 — Library
- G1440 — Mediation Chapel
- G1450 — Reflection Space
- G1460 — Chapel
- G1470 — Shrine
- G1480 — Confessional Space
- G1490 — Tabernacle
- G1500 — Choir Loft
- G1510 — Marriage Sanctuary
- G1520 — Mental Health Quiet Room

- G1530 — Bone Densitometry Room
- G1540 — CT Simulator Room
- G1550 — Head Radiographic Room
- G1560 — Mobile Imaging System Alcove
- G1570 — MRI System Component Room
- G1580 — PET/CT Simulator Room
- G1590 — Radiographic Room
- G1600 — Stereotactic Mammography Room
- G1610 — Ultrasound/Optical Coherence Tomography Room
- G1620 — Angiographic Control Room
- G1630 — Angiographic Procedure Control Area
- G1640 — Silver Collection Area
- G1650 — Computer Image Processing Area
- G1660 — CT Control Area
- G1670 — Image Quality Control Room
- G1680 — X-Ray, Plane Film Storage Space
- G1690 — MRI Control Room
- G1700 — MRI Viewing Room
- G1710 — Radiographic Control Room
- G1720 — Tele-Radiology/Tele-Medicine Room
- G1730 — Radiation Diagnostic and Therapy Spaces
- G1740 — Health Physics Laboratory
- G1750 — Linear Accelerator Entrance Maze, Healthcare
- G1760 — Radioactive Waste Storage Room, Healthcare
- G1770 — Nuclear Medicine Scanning Room
- G1780 — Patient Dose/Thyroid Uptake Room
- G1790 — Radiopharmacy
- G1800 — Radiation Therapy, Mold Fabrication Shop

- G1810 — Hearth and Lung Diagnostic and Treatment Spaces
- G1820 — Cardiac Catheter Instrument Room
- G1830 — Cardiac Catheter Control Room
- G1840 — Cardiac Electrophysiology Room
- G1850 — Echocardiograph Room
- G1860 — Extended Pulmonary Function Testing Laboratory
- G1870 — Pacemaker ICD Interrogation Room
- G1880 — Procedure Viewing Area
- G1890 — Pulmonary Function Treadmill Room
- G1900 — Respiratory Therapy Clean-up Room
- G1910 — General Diagnostic Procedure and Treatment Spaces
- G1920 — Endoscopy/Gastroenterology Spaces
- G1930 — Clinical Laboratory Spaces
- G1940 — Pharmacy Spaces
- G1950 — Rehabilitation Spaces
- G1960 — Medical Research and Development Spaces
- G1970 — Chemistry Laboratories
- G1980 — Physical Sciences Laboratories
- G1990 — Earth and Environmental Sciences Laboratories
- G2000 — Psychology Laboratories
- G2010 — Dry Laboratories
- G2020 — Wet Laboratories
- G2030 — Biosciences Laboratories
- G2040 — Astronomy Laboratories
- G2050 — Forensics Laboratories
- G2060 — Bench Laboratories
- … **(Refer to the full standard implementation schema for other classifications)**

C.2. TransferSpace

C.2.1. TransferSpaceCategoryType

- C1000 — Door
- C1010 — Vestibule
- C1020 — Sally port
- A1000 — Vestibule
- A1020 — Gate

C.2.2. TransferSpaceFunctionType

- C1000 — Door
- C1010 — Vestibule
- C1060 — Sally port
- A1000 — Entry Vestibule
- A1010 — Exterior door
- A1020 — Gate
- A1030 — Emergency door

D

ANNEX D (INFORMATIVE) REVISION HISTORY

ANNEX D (INFORMATIVE) REVISION HISTORY

DATE	RELEASE	EDITOR	PRIMARY CLAUSES MODIFIED	DESCRIPTION
2026-06-03	0.1	Taehoon Kim	all	Initial draft
2026-08-01	0.2	Taehoon Kim	Clause 7, 8, Annex A	Add ATS
2026-08-18	0.5	Taehoon Kim	all	Reflecting the feedback from the editorial meeting